

A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

Tesina Finale di

Programmazione di Interfacce Grafiche e Dispositivi Mobili

Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2025-2026

Dipartimento di Ingegneria

docente

Prof. Luca Grilli

PollageSF

applicazione desktop JavaFX

Studenti

364672	**Romeo**	**Miscio**	romeo.miscio@studenti.unipg.it
367169	**Paolo**	**Ferrari**	paolo.ferrari@studenti.unipg.it

Ultimo aggiornamento: 1 novembre 2026

Indice

1 Descrizione del problema **2**
1.1 Street Fighter - il videogioco . . . 2
1.2 L'applicazione PollageSF . . . 2

2 Specifica dei requisiti **4**

3 Progetto **6**
3.1 Architettura del sistema software e relazioni UML . . . 6
3.2 Model . . . 8
3.2.1 Coordinamento del mondo e telecamera (`GameModel`) . . . 8
3.2.2 Gerarchia dei lottatori (`Player`, `Ryu`, `Ken`) . . . 8
3.2.3 Sistema polimorfo delle mosse (`Move`, `MeleeMove`, `RangedMove`) . . . 9
3.2.4 Collisioni AABB e meccanica del parry (`Hitbox`) . . . 10
3.3 View . . . 11
3.3.1 Gestione della scena e proiezione della telecamera (`GameView`) . . . 11
3.3.2 Rendering pixel art e frame caching (`PlayerRenderer`) . . . 12
3.3.3 Interfaccia utente in partita (`HUDView`) . . . 13
3.4 Controller . . . 13
3.4.1 Game loop a passo fisso e macchina a stati (`GameController`) . . . 13
3.4.2 Gestione periferiche hardware (`InputManager`) . . . 14
3.4.3 Gestione disaccoppiata del comparto audio (`SoundManager` e `SoundEvent`) 15
3.5 Problemi riscontrati . . . 16
3.5.1 Avvio dell'applicazione tramite la classe `Main.java` . . . 16
3.5.2 Rendering dello sprite sheet . . . 17
3.5.3 Gestione del palette swap e iniezione delle skin . . . 18
3.5.4 Disaccoppiamento della nascita dei proiettili (*Deferred Spawning*) . . . 18
3.5.5 Sfarfallio e compenetrazione dei corpi nel pavimento . . . 18

4 Conclusioni e sviluppi futuri **19**

5 Bibliografia **20**

6 Appendice **21**
6.1 Mappatura dei comandi controller (Gamepad) . . . 21
6.2 Tabella dei frame data e statistiche dei lottatori . . . 22

1 Descrizione del problema

L'obiettivo di questo lavoro è lo sviluppo di un'applicazione desktop, denominata **PollageSF**, che realizza una versione semplificata ma ingegneristicamente rigorosa del celebre videogioco picchiaduro *Street Fighter* [1]. L'applicazione è stata interamente progettata e implementata utilizzando il linguaggio Java e la tecnologia **JavaFX** [2], scelta per favorire un'ampia compatibilità su diversi sistemi operativi e per garantire una gestione strutturata del grafo di scena (*Scene Graph*), del rendering grafico hardware-accelerato e delle animazioni in tempo reale.

Di seguito viene presentata una sintetica panoramica del videogioco arcade originale *Street Fighter*, seguita da una descrizione dettagliata delle caratteristiche funzionali e di gameplay della versione **PollageSF** realizzata.

1.1 Street Fighter - il videogioco

Street Fighter è un celebre videogioco arcade di combattimento a incontri in tempo reale sviluppato da Capcom, strutturato su duelli uno contro uno all'interno di un'arena bidimensionale a scorrimento orizzontale [1]. Ciascun giocatore seleziona un lottatore dal roster disponibile, ognuno caratterizzato da attributi fisici specifici, un set unico di attacchi normali corpo a corpo (pugni rapidi, ganci pesanti, calci alti e bassi), mosse speciali a distanza (come il celebre proiettile energetico *Hadouken*) e tecniche difensive.

L'obiettivo primario di ogni incontro consiste nell'azzerare la barra della salute (*Health Bar*) dell'avversario prima dello scadere del tempo regolamentare. La profondità strategica e competitiva del titolo risiede in tre fattori fondamentali:

- La gestione accurata delle distanze (*footsies* e *spacing*) e la previsione delle intenzioni dell'avversario;
- La conoscenza delle finestre temporali delle mosse (*Startup*, *Active* e *Recovery frames*) per punire gli attacchi andati a vuoto;
- Il controllo dello spazio dell'arena per chiudere l'avversario all'angolo (*corner pressure*) o neutralizzare l'offensiva nemica tramite parate tempestive (*Parry*).

1.2 L'applicazione PollageSF

PollageSF riproduce le meccaniche fondamentali del picchiaduro bidimensionale all'interno di un'applicazione desktop progettata secondo il pattern architetturale **Model-View-Controller (MVC)**. Il sistema è pensato per offrire un'esperienza multigiocatore locale immediata e reattiva tramite l'utilizzo simultaneo di gamepad esterni gestiti a basso livello mediante la libreria *Jamepad* (basata su SDL2) [3].

Il flusso operativo dell'applicazione accompagna gli utenti attraverso una sequenza strutturata di menu interattivi prima di avviare lo scontro:

1. **Menu di Connessione Controller (*Drop-in*):** L'applicazione rileva in tempo reale i gamepad collegati via USB o Bluetooth. I giocatori si uniscono alla sessione premendo un tasto d'azione, con supporto sia per la modalità a due giocatori umani sia per la configurazione a giocatore singolo.
2. **Menu di Selezione del Lottatore e delle Skin:** Una griglia interattiva consente ai giocatori di muovere indipendentemente i propri cursori (`P1` rosso e `P2` blu) tra i personaggi del roster (*Ryu*, *Ken*). Durante la selezione, vengono mostrate in tempo reale le anteprime animate dei lottatori scelti, potendo decidere il colore della skin (*skin*) tramite i tasti dorsali `L1` e `R1`.

3. **Menu di Selezione dell'Arena:** I giocatori scelgono lo scenario di combattimento tra dieci mappe differenti (tra cui *UniPG*, *Fight Club*, *Paradise & Hell*, *Villaggio incantato*), ciascuna dotata di una propria immagine panoramica ad alta risoluzione e di una specifica altezza prospettica del piano di calpestio (`groundLevelRatio`).

Una volta avviato il match, i due contendenti si affrontano in un'arena più ampia della singola schermata visibile. Una telecamera virtuale dinamica segue l'azione calcolando il punto medio tra i lottatori e applicando una zona morta centrale (*deadzone*) combinata con un'interpolazione lineare (*LERP*) per garantire movimenti di inquadratura fluidi. Il sistema di combattimento integra attacchi in piedi e accovacciati (*crouch*), proiettili energetici e una meccanica avanzata di difesa che distingue tra parata parziale (con riduzione progressiva del danno) e *Parry perfetto* (che azzera il danno subito e stordisce temporaneamente l'attaccante).

2 Specifica dei requisiti

L'applicazione **PollageSF** è stata progettata e realizzata per soddisfare i seguenti requisiti funzionali e non funzionali, espressi in forma puntuale secondo lo standard di progetto:

1. **Separazione srchitetturale MVC:** L'intero sistema software deve rispettare rigorosamente il pattern *Model-View-Controller*, disaccoppiando la logica di simulazione fisica (`application.Model`) dalla rappresentazione grafica JavaFX (`application.View`) attraverso interfacce a sola lettura (`IReadOnlyGameModel` e `IReadOnlyPlayer`).

2. **Game loop a passo fisso con accumulatore:** L'aggiornamento del motore fisico deve essere governato da un ciclo basato su `AnimationTimer` che utilizza un accumulatore temporale (`physicsAccumulator`) e un passo costante (`TIME_PER_TICK`), garantendo una simulazione deterministica indipendente dalla frequenza di rendering (`targetFps`).

3. **Gestione multi-gamepad e drop-in dinamico:** Il sistema deve supportare l'input simultaneo e privo di latenza da due controller tramite la libreria SDL2/Jamepad, offrendo un menu iniziale di accoppiamento alla pressione di un tasto.

4. **Sistema di sicurezza per disconnessione controller:** Durante il combattimento, l'applicazione deve monitorare costantemente lo stato fisico dei gamepad e, in caso di scollegamento accidentale, congelare immediatamente la fisica di gioco mostrando un overlay di emergenza con sfocatura gaussiana (`GaussianBlur`) fino alla riconnessione del dispositivo.

5. **Selezione personaggi con anteprima animata e skin:** Il menu di scelta del lottatore deve consentire la selezione indipendente per Giocatore 1 e Giocatore 2, visualizzando lo sprite animato in tempo reale e permettendo la personalizzazione della palette cromatica (*skin*) tramite i tasti dorsali `L1` e `R1`.

6. **Selezione arena con telecamera UI scorrevole:** Il menu di selezione della mappa deve presentare una griglia di anteprime con bordi arrotondati, inseguimento automatico dello scroll verticale (`ScrollPane`) sul nodo selezionato e aggiornamento dinamico dello sfondo in anteprima.

7. **Gestione della fisica e collisioni:** Il motore fisico deve gestire salti e forza di gravità, applicando un moltiplicatore di gravità maggiorato ($2.5\times$) qualora il giocatore rilasci anticipatamente il pulsante di salto durante la fase di salita. Devono essere rilevate le intersezioni tra Hitbox d'attacco e Hurtbox corporea per validare il danno, impedendo inoltre la compenetrazione dei corpi tra i due giocatori mediante una forza di reazione repulsiva.

8. **Sistema di difesa dinamico e parry perfetto:** La parata frontale deve ridurre il danno in proporzione al fotogramma di parata raggiunto; se l'impatto avviene sull'ultimo frame dell'animazione di blocco, deve attivarsi un *Parry perfetto* che annulla il danno e infligge uno stato di stordimento (`stun`) all'attaccante.

9. **Telecamera dinamica con deadzone e LERP:** L'inquadratura dell'arena deve seguire il punto medio tra i due lottatori, attivando lo scorrimento solo al superamento delle soglie del 25% e 75% dello schermo e applicando un'interpolazione lineare morbida (cameraSpeed = 0.08).

10. **Scaling nearest-neighbor e caching delle texture:** Per preservare la nitidezza della pixel art a qualsiasi risoluzione dello schermo, il renderer deve ingrandire i fotogrammi tramite campionamento *Nearest-Neighbor* via software, salvando le immagini risultanti in una struttura di cache (`HashMap`) indicizzata anche per palette di colore.

11. **Interfaccia HUD e risoluzione indipendente:** L'interfaccia utente in partita (`HUDView`) deve mostrare barre della salute reattive (con riduzione speculare verso il centro dello schermo) e il timer di gioco, scalando proporzionalmente tutti gli elementi rispetto all'altezza della finestra.

12. **Gestione pausa e fine partita:** Il gioco deve consentire la messa in pausa tramite controller con navigazione dei pulsanti via levetta analogica e gestire la schermata di vittoria finale al raggiungimento di 0 punti vita da parte di uno dei contendenti.

13. **Sistema audio disaccoppiato (SFX e BGM):** Il sistema deve riprodurre effetti sonori a bassa latenza per le azioni di combattimento e musiche di sottofondo con memoria della posizione di riproduzione, mantenendo il `GameModel` disaccoppiato dalle librerie multimediali tramite una coda di eventi (`SoundEvent`).

3 Progetto

Nel presente capitolo viene descritta nel dettaglio la progettazione del gioco **PollageSF**, analizzando l'architettura complessiva, le responsabilità dei singoli package (`Model`, `View`, `Controller`) e le soluzioni adottate per risolvere le principali problematiche tecniche emerse durante lo sviluppo.

3.1 Architettura del sistema software e relazioni UML

Il sistema software è stato progettato adottando rigorosamente il pattern architetturale **Model-View-Controller (MVC)**, al fine di garantire un elevato grado di modularità, manutenibilità ed estendibilità del codice sorgente, nonché il rispetto dell'*Acyclic Dependencies Principle* (ADP). La separazione delle responsabilità e le relazioni strutturali tra i package (illustrate globalmente in Figura 1) sono così definite:

- **Modulo model (`pgsf.model`):** Contiene la logica di dominio, lo stato della partita, la cinematica dei corpi, la gestione delle animazioni a livello logico, la coda degli eventi sonori (`SoundEvent`) e il motore delle collisioni. Il Model non importa alcun nodo grafico né multimediale della libreria JavaFX (come `Pane`, `ImageView` o `MediaPlayer`), risultando completamente indipendente dalla tecnologia di presentazione.
- **Modulo view (`pgsf.view`):** Si occupa esclusivamente della rappresentazione visiva su schermo. Riceve in ingresso dal Controller un riferimento al Model tipizzato attraverso le interfacce di sola lettura `IReadOnlyGameModel`, `IReadOnlyPlayer` e `IReadOnlyProjectile`. In questo modo la View interroga lo stato del mondo per disegnare i fotogrammi, ma le è impedito a livello di compilazione invocare metodi mutatori.
- **Modulo controller (`pgsf.controller`) e Utilities (`pgsf.utils`):** Costituisce il coordinatore dell'applicazione. Gestisce il ciclo di vita della finestra (`Stage` e `Scene`), mantiene attivo il *Game Loop* principale, interroga le periferiche hardware tramite `InputManager`, governa il sottosistema audio (`SoundManager`) e invoca a ogni tick prima l'aggiornamento logico (`model.update(inputManager)`), poi il consumo degli eventi sonori (`model.pollSoundEvents()`) e infine il rendering grafico (`view.render(model)`).

Dal punto di vista dell'ingegneria del software, il diagramma delle classi in Figura 1 mette in evidenza quattro tipologie di relazioni UML fondamentali che garantiscono il disaccoppiamento:

1. **Realizzazione e inversione delle dipendenze (DIP):** Le classi di dominio `GameModel`, `Player` e `Projectile` realizzano rispettivamente le interfacce `IReadOnlyGameModel`, `IReadOnlyPlayer` e `IReadOnlyProjectile`. Analogamente, nel package `model` è definita l'interfaccia `IReadOnlyInput`, realizzata concretamente da `InputManager` nel package `controller`: grazie a questa inversione, `GameModel` dipende esclusivamente da un'astrazione appartenente al proprio stesso package, senza mai conoscere il `controller`.
2. **Composizione (`*-`):** Il `GameController` detiene il ciclo di vita esclusivo di `GameModel` e `GameView`. A sua volta, `GameModel` compone le due istanze di `Player`, mentre `GameView` compone i due `PlayerRenderer` e l'`HUDView`.
3. **Aggregazione (`o-`):** Il `GameModel` aggrega dinamicamente le istanze di `Projectile` in volo e gli eventi `SoundEvent` nella coda temporanea `soundQueue`, mentre `CharacterData` aggrega l'interfaccia funzionale `CharacterFactory`.
4. **Dipendenza d'uso (`..>`):** Le classi della View (`GameView`, `PlayerRenderer`, `HUDView`) presentano unicamente frecce tratteggiate di dipendenza verso le interfacce `IReadOnly*` del Model, certificando l'assenza di accoppiamento verso le implementazioni concrete.

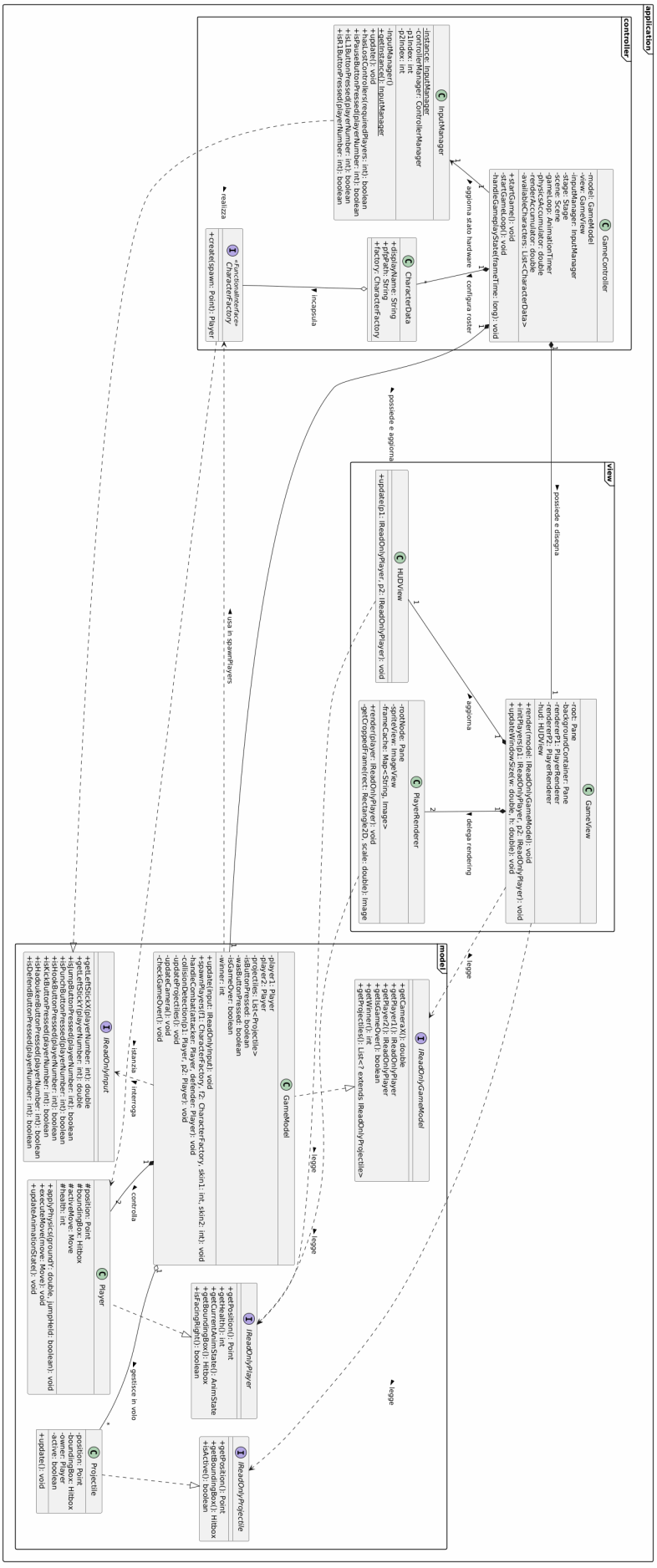


Figura 1: Diagramma UML completo dell'architettura Model-View-Controller del sistema PollageSF e delle relazioni inter-package.

3.2 Model

Il package `application.Model` racchiude il cuore computazionale e fisico del picchiaduro. Di seguito vengono analizzate le classi fondamentali che lo compongono.

3.2.1 Coordinamento del mondo e telecamera (`GameModel`)

La classe `GameModel` (che implementa `IReadOnlyGameModel`) mantiene i riferimenti ai due contendenti (`player1` e `player2`), alla lista dinamica dei proiettili attivi (`List<Projectile> projectiles`) e alle dimensioni logiche dell'arena (`worldWidth` e `currentGroundLevel`). Tutte le entità vivono in un sistema di coordinate assolute espresso tramite la classe `Point(x, y)`.

Ad ogni invocazione del metodo `update(InputManager input)`, il modello esegue in sequenza:

1. L'aggiornamento dei contatori temporali delle mosse attive (`updateTicks()`);
2. L'integrazione della fisica verticale e della gravità (`applyPhysics()`) per entrambi i giocatori;
3. La lettura degli input di movimento, salto, postura accovacciata (`crouch`), parata e attacco (solo se il giocatore non si trova nello stato di stordimento `isStunned()`), rilevando i fronti di salita dei pulsanti tramite flag di memoria (`wasP1PunchHeld`, `wasP1JumpHeld`, ecc.) per evitare ripetizioni involontarie tenendo premuto il tasto;
4. L'avanzamento dei proiettili in volo e la verifica dell'impatto con il bersaglio;
5. La risoluzione del combattimento corpo a corpo (`handleCombat`) in entrambe le direzioni;
6. L'applicazione dei vincoli spaziali (`keepPlayerInBounds` e `collisionDetection`);
7. Il calcolo della posizione della telecamera (`cameraX`) e la chiusura dei giocatori all'interno dell'inquadratura visibile (`keepPlayerOnScreen`).

Particolarmente significativa è la logica matematica che governa la telecamera orizzontale. Dato il punto medio assoluto tra i due lottatori:

$$X_{\text{mid}} = \frac{X_{P1} + X_{P2}}{2} \tag{1}$$

viene calcolata la sua proiezione rispetto all'inquadratura corrente ($X_{\text{screen}} = X_{\text{mid}} - \text{cameraX}$). Definiti i margini della *Deadzone* centrale al 25% ($T_L = 0.25 \cdot W_{\text{win}}$) e al 75% ($T_R = 0.75 \cdot W_{\text{win}}$) della larghezza della finestra, la posizione target della telecamera targetCameraX viene modificata solo quando il fulcro oltrepassa tali soglie, venendo poi limitata (*clamping*) nell'intervallo $[0, \text{worldWidth} - W_{\text{win}}]$. Infine, per evitare scatti bruschi, viene applicata un'interpolazione lineare (*LERP*) ad ogni tick:

$$\text{cameraX}_{k+1} = \text{cameraX}_k + (\text{targetCameraX} - \text{cameraX}_k) \cdot \alpha \quad \text{con } \alpha = 0.08 \tag{2}$$

3.2.2 Gerarchia dei lottatori (`Player`, `Ryu`, `Ken`)

La classe `Player` modella un generico combattente implementando l'interfaccia `IReadOnlyPlayer`. Essa gestisce:

- **Cinematica e Salto Adattivo:** Nel metodo `applyPhysics(double groundLevelY, boolean isJumpHeld)`, la velocità verticale `velocityY` viene incrementata ad ogni tick del valore `gravity`. Se tuttavia il personaggio sta salendo (velocityY < 0) ma l'utente ha già rilasciato il tasto di salto (`!isJumpHeld`), il sistema applica una gravità amplificata (velocityY += gravity $\cdot$ 2.5), permettendo di eseguire salti bassi tattici (*short hop*) o salti completi in base alla durata della pressione.

- **Accovacciamento e Ricalcolo Hurtbox:** Il metodo `setCrouching(boolean)` riduce l'altezza del personaggio al 60% dell'altezza normale (targetHeight $= 0.6 \cdot$ normalHeight) e trasla simultaneamente la coordinata Y verso il basso della differenza $\Delta H =$ normalHeight$-$ targetHeight, garantendo che i piedi rimangano saldamente ancorati al pavimento mentre la testa scende per schivare colpi alti.

- **Macchina a Stati delle Animazioni:** Il metodo `updateAnimationState()` determina a ogni tick quale valore dell'enumerazione `AnimState` assegnare al giocatore, seguendo una rigorosa gerarchia di priorità:

  1. *Stordimento (`isStunned()`):* priorità assoluta; congela il lottatore nel fotogramma corrente;
  2. *Danno subito (`isHurt`):* blocca il personaggio nell'animazione `HURT_RIGHT/LEFT` (o `HURT_CROUCH`) per l'intera durata temporale prevista dai frame (`frameCount * speedNs`);
  3. *Cambio di direzione (`isTurning`):* esegue l'animazione `TURN` per 150 ms quando il giocatore inverte l'orientamento a terra;
  4. *Mossa d'attacco attiva (`activeMove != null`):* imposta lo stato dettato dalla mossa in corso (`PUNCH`, `HOOK`, `KICK`, `HADOUKEN`, `PUNCH_CROUCH`);
  5. *Difesa (`isDefending`):* seleziona `BLOCK` o `CROUCH_BLOCK`;
  6. *Postura e Movimento (`isCrouching`, `!isGrounded`, `isMoving`):* seleziona rispettivamente `CROUCH`, `JUMP` o `WALK`;
  7. *Stato di riposo:* imposta `MENU_IDLE` (se nel menu di selezione) oppure `IDLE_RIGHT/LEFT`.

- **Scaling dinamico universale:** Attraverso i metodi `saveBaseStats()` e `updateDynamicScale(double windowScale)`, il giocatore conserva le statistiche fisiche di base (velocità, gravità, forza di salto, dimensioni logiche della hitbox e portata dei colpi) e le moltiplica dinamicamente per il fattore di scala della finestra, garantendo che la fisica si comporti in modo identico a qualsiasi risoluzione video.

Le sottoclassi concrete, come `Ryu` e `Ken`, estendono `Player` definendo nel proprio costruttore le statistiche specifiche (ad esempio `Ryu` ha punchDamage $= 10.0$, mentre `Ken` ha un attacco rapido da 9.0), il percorso al manifesto JSON delle texture (`TextureAtlas`), la larghezza calibrata della hurtbox (width $=$ defaultFrame.getWidth() $\cdot 0.70 \cdot$ renderScale), la mappa completa `Map<AnimState, AnimData> animations` e la lista delle 7 palette cromatiche disponibili (`availableSkins`).

3.2.3 Sistema polimorfo delle mosse (`Move`, `MeleeMove`, `RangedMove`)

Per evitare la proliferazione di variabili booleane all'interno della classe `Player`, le azioni offensive sono state modellate mediante polimorfismo:

- **`Move` (Classe Astratta):** Memorizza il riferimento al lottatore che esegue l'azione (`Player user`), l'istante di avvio in nanosecondi (`startTimeNs`) e il flag di sicurezza `hasHit` (che impedisce a un singolo attacco di infliggere danno in più tick consecutivi). Espone i metodi astratti `isFinished()`, `getActiveHitbox()`, `getDamage()` e `getAnimState()`.

- **`MeleeMove` (Attacchi Corpo a Corpo):** Implementa pugni, ganci, calci e colpi accovacciati. La durata complessiva della mossa è calcolata confrontando il tempo trascorso $\Delta t =$ System.nanoTime() $-$ startTimeNs con il prodotto frameCount $\cdot$ speedNs estratto da `AnimData`. Il metodo `getActiveHitbox()` restituisce `null` durante i fotogrammi di

caricamento (*Startup*) e genera dinamicamente l'oggetto `Hitbox` davanti al giocatore solo quando viene raggiunto l'ultimo fotogramma dell'animazione:

$$\Delta t \geq (\text{frameCount} - 1) \cdot \text{speedNs} \tag{3}$$

- **`RangedMove` (Attacchi a Distanza):** Gestisce mosse come l'*Hadouken*, sincronizzando l'istanziazione dell'entità `Projectile` con il fotogramma esatto in cui il personaggio estende le mani in avanti, assicurando tramite un flag interno (`hasFired`) che venga generato un solo proiettile per ogni esecuzione.

3.2.4 Collisioni AABB e meccanica del parry (`Hitbox`)

La classe `Hitbox` incapsula un rettangolo orientato agli assi $(x, y, \text{width}, \text{height})$ e implementa il test di intersezione geometrica **AABB (*Axis-Aligned Bounding Box*)** nel metodo `intersects(Hitbox other)`:

$$\text{intersects} = (x_1 + w_1 \geq x_2) \wedge (x_1 \leq x_2 + w_2) \wedge (y_1 + h_1 \geq y_2) \wedge (y_1 \leq y_2 + h_2) \tag{4}$$

Tale primitiva geometrica viene impiegata in due contesti distinti:

1. **Anti-compenetrazione tra corpi (`collisionDetection`):** Se le hurtbox di `player1` e `player2` si sovrappongono lungo l'asse orizzontale, viene calcolata la profondità di compenetrazione:
$$\text{overlap} = \left(\frac{w_1}{2} + \frac{w_2}{2}\right) - |C_{x1} - C_{x2}| \tag{5}$$
Se overlap > 0, entrambi i giocatori vengono respinti simmetricamente verso l'esterno di una distanza pari a overlap/2.

2. **Risoluzione degli Attacchi e Parry (`handleCombat`):** Quando la `Hitbox` attiva di una mossa interseca la `boundingBox` del difensore, il modello verifica se il difensore sta parando (`isDefending()`) ed è rivolto verso l'attaccante (`isFacingAttacker`). In caso affermativo, viene calcolato il fotogramma teorico di parata in cui si trova il difensore a partire dall'istante di inizio blocco t_{block}:
$$F_{\text{block}} = \left\lfloor \frac{t_{\text{now}} - t_{\text{block}}}{\text{speedMs}_{\text{block}}} \right\rfloor \tag{6}$$
Se $F_{\text{block}} \geq N_{\text{frames}} - 1$ (ossia l'impatto coincide con il completamento dell'animazione di parata), si attiva il **Parry Perfetto**: il difensore subisce 0 danni e l'attaccante viene stordito per parryStunDuration (ms). In caso contrario, si verifica una **Parata Parziale** con danno ridotto secondo la legge:
$$\text{Danno Finale} = \text{Danno Base} \cdot \left(1 - \frac{F_{\text{block}} + 1}{N_{\text{frames}}}\right) \tag{7}$$

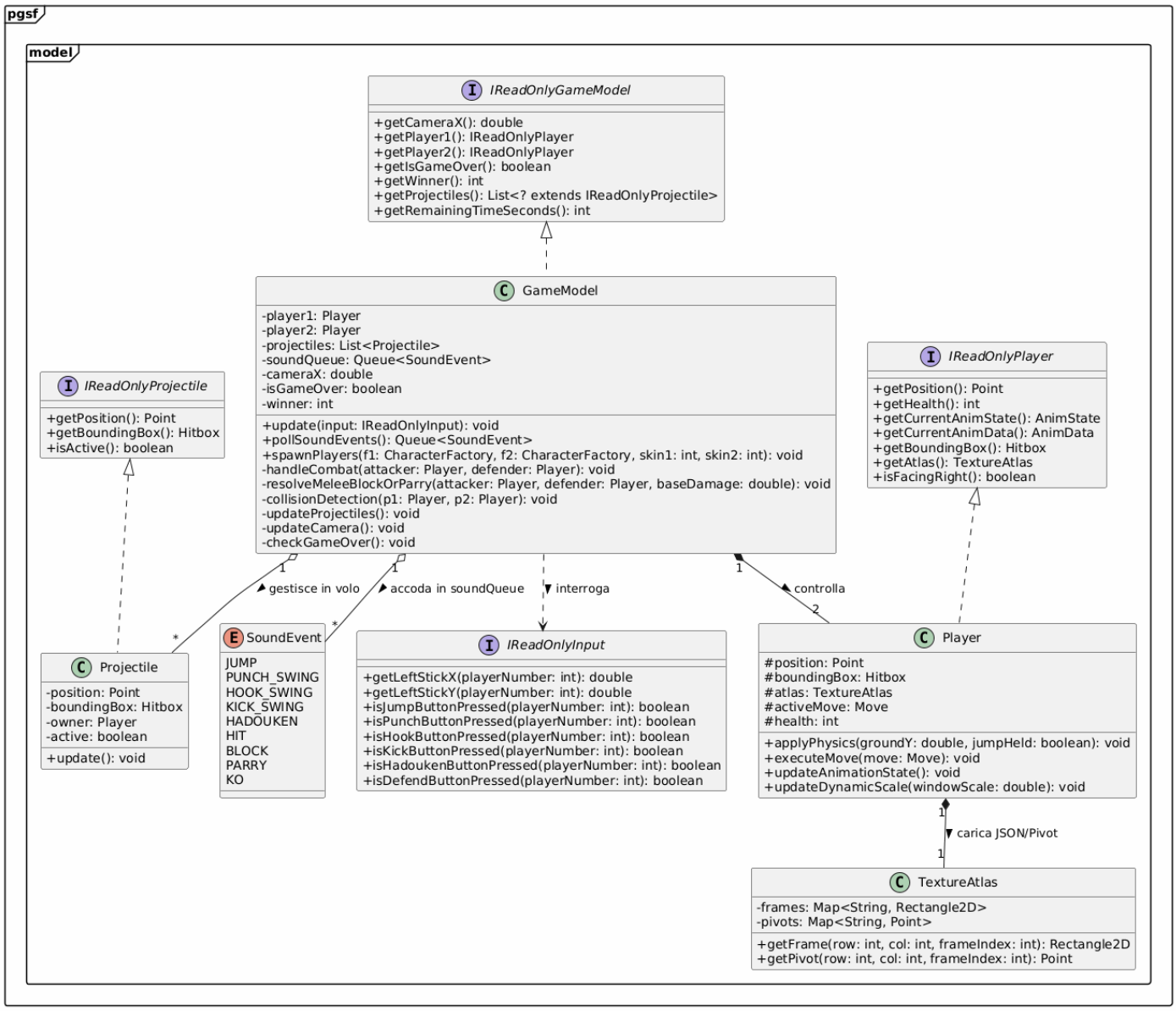


Figura 2: Dettaglio del diagramma delle classi per il package `pgsf.model`.

3.3 View

Il package `application.View` traduce lo stato logico del Model in elementi visivi JavaFX, occupandosi della telecamera, degli sprite e dell'interfaccia utente.

3.3.1 Gestione della scena e proiezione della telecamera (`GameView`)

La classe `GameView` gestisce il contenitore radice (`Pane root`) sul quale vengono impilati i diversi livelli visivi: lo sfondo (`backgroundContainer` dotato di maschera di ritaglio `Rectangle clip`), i nodi grafici dei due lottatori (`PlayerRenderer`) e, in primo piano, l'interfaccia `HUDView` e l'overlay di fine partita (`victoryLayer`).

Nel metodo `render(IReadOnlyGameModel model)`, la trasformazione dalle coordinate assolute del mondo alle coordinate relative dello schermo viene realizzata traslando orizzontalmente i contenitori della quantità $-$cameraX:

```
this.backgroundContainer.setLayoutX(-camX);
```

```
rendererP1.getNode().setLayoutX(-camX);
rendererP2.getNode().setLayoutX(-camX);
```

Inoltre, il metodo `updateWindowSize(double newWidth, double newHeight)` risolve il problema dell'adattamento a diverse risoluzioni separando lo scaling dello sfondo da quello degli elementi di gioco:

- Lo **sfondo** viene scalato mediante il fattore currentScale = $\max(W_{\text{new}}/W_{\text{orig}}, H_{\text{new}}/H_{\text{orig}})$, assicurando la copertura integrale della finestra senza bande vuote e ancorando l'immagine al bordo inferiore (offsetY = $H_{\text{new}} - H_{\text{bg}}$);
- I **personaggi e l'HUD** utilizzano invece un fattore di scala normalizzato dipendente esclusivamente dall'altezza della finestra (gameScale = $H_{\text{new}}/720.0$), garantendo proporzioni costanti indipendentemente dal formato dell'immagine dell'arena scelta.

3.3.2 Rendering pixel art e frame caching (`PlayerRenderer`)

La classe `PlayerRenderer` gestisce l'animazione e il disegno del singolo lottatore a partire dal suo sprite sheet (`atlasImage`). Ad ogni chiamata di `render(IReadOnlyPlayer player)`, la classe verifica se lo stato `AnimState` è cambiato (azzerando in tal caso l'indice `currentFrame`), fa avanzare il fotogramma quando il tempo trascorso supera `currentData.speedNs` (gestendo sia le animazioni cicliche `loop = true` tramite operatore modulo, sia quelle singole tramite `Math.min`) e recupera dal `TextureAtlas` il rettangolo di ritaglio `Rectangle2D frameRect`.

Poiché il ridimensionamento nativo delle immagini in JavaFX può introdurre artefatti di interpolazione bilineare che sfocano la pixel art, è stato implementato nel metodo `getCroppedFrame(Rectangle2D rect, double scale)` un algoritmo di **campionamento Nearest Neighbor via software** basato su `PixelReader` e `PixelWriter`. Per ogni coordinata (x, y) dell'immagine di destinazione di dimensioni $(W_{\text{dst}}, H_{\text{dst}}) = (\text{round}(W_{\text{src}} \cdot s), \text{round}(H_{\text{src}} \cdot s))$, il colore ARGB viene prelevato dalla coordinata sorgente sull'atlante:

$$X_{\text{src}} = \left\lfloor X_{\min} + \frac{x}{s} \right\rfloor, \qquad Y_{\text{src}} = \left\lfloor Y_{\min} + \frac{y}{s} \right\rfloor \tag{8}$$

Per azzerare l'overhead computazionale che deriverebbe dal ricalcolo matriciale dei pixel a 60 FPS, ogni `WritableImage` generata viene memorizzata in una mappa di cache `Map<String, Image> frameCache` tramite il metodo `computeIfAbsent`, utilizzando una chiave univoca composta da `"minX_minY_width_height_scale"`. In questo modo il ritaglio viene calcolato una sola volta al primo utilizzo del fotogramma e restituito in tempo costante $O(1)$ per tutto il resto della partita.

Infine, per risolvere il disallineamento geometrico dovuto alle dimensioni variabili dei riquadri di ritaglio $(W_{\text{frame}}, H_{\text{frame}})$ all'interno del manifesto JSON (ad esempio durante l'estensione del braccio nei pugni o l'oscillazione verticale della respirazione in `IDLE`), `TextureAtlas` associa a ciascun fotogramma un punto di ancoraggio $\mathbf{P} = (\text{pivot}_x, \text{pivot}_y)$, con fallback automatico posto al centro della base inferiore del corpo standard $(W_{\text{base}}/2, H_{\text{frame}}) = (23.5, H_{\text{frame}})$. Partendo dalle coordinate base $(p_x, p_y) = (X_{\text{player}} - \text{offset}_x, Y_{\text{player}} - \text{offset}_y)$, il posizionamento finale su schermo applica la compensazione verticale ai piedi e la compensazione orizzontale simmetrica per l'inversione speculare (`setScaleX(-1)` quando il lottatore è rivolto a sinistra):

$$p'_y = p_y + \Big(H_{\text{base}} - \text{pivot}_y\Big) \cdot s, \qquad p'_x = \begin{cases} p_x - \Big(\text{pivot}_x - \frac{W_{\text{base}}}{2}\Big) \cdot s & \text{se rivolto a destra} \\ p_x - \Big[(W_{\text{frame}} - \text{pivot}_x) - \frac{W_{\text{base}}}{2}\Big] \cdot s & \text{se rivolto a sinistra} \end{cases} \tag{9}$$

3.3.3 Interfaccia utente in partita (`HUDView`)

La classe `HUDView` disegna nella parte superiore dello schermo i nomi dei giocatori, le barre della salute (`p1HealthFill`, `p2HealthFill` sovrapposte ai rispettivi sfondi rosso scuro), le barre secondarie del potere speciale e il contatore centrale del tempo. Nel metodo `update(IReadOnlyPlayer p1, IReadOnlyPlayer p2)`, mentre la barra della vita del Giocatore 1 si riduce semplicemente accorciando la propria larghezza da destra verso sinistra, per il Giocatore 2 viene applicata una traslazione compensativa sulla coordinata X:

$$W_{P2} = W_{\text{bar}} \cdot \frac{\text{HP}_{P2}}{\text{HP}_{\text{max},P2}}, \qquad X_{P2} = X_{\text{bg},P2} + (W_{\text{bar}} - W_{P2}) \tag{10}$$

ottenendo l'effetto classico dei picchiaduro arcade in cui entrambe le barre della salute si svuotano simmetricamente verso il timer centrale.

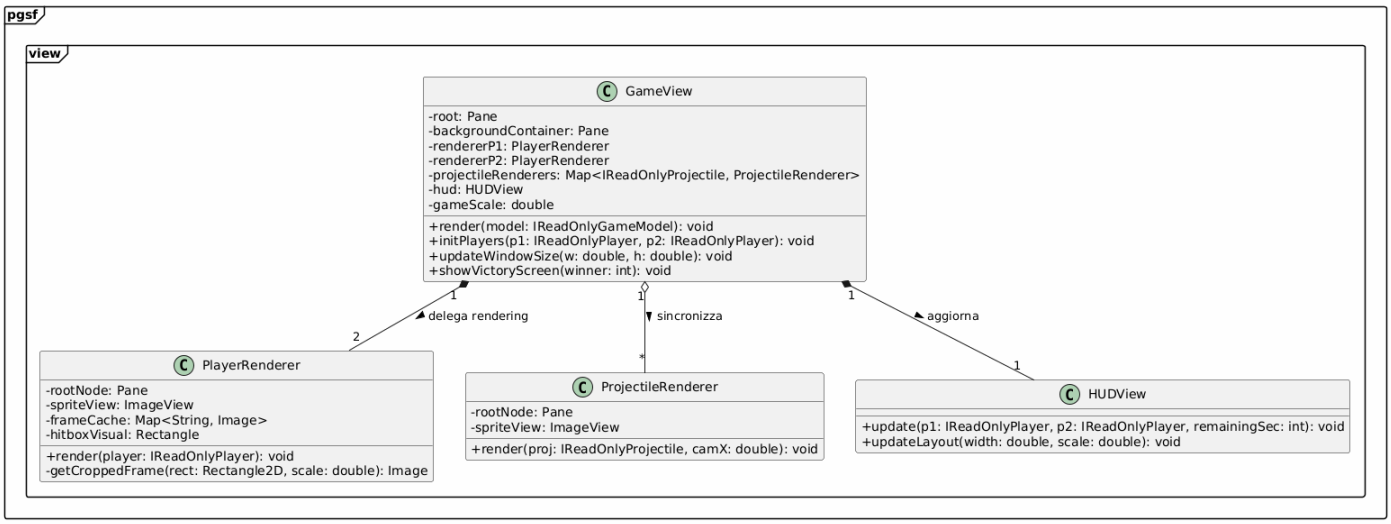


Figura 3: Dettaglio del diagramma delle classi per il package `pgsf.view`.

3.4 Controller

Il package `application.Controller` coordina l'esecuzione temporale, le periferiche di input e la navigazione tra gli stati dell'applicazione.

3.4.1 Game loop a passo fisso e macchina a stati (`GameController`)

La classe `GameController` costruisce nel metodo `startGame()` uno `StackPane` stratificato (`mainRoot`) che impila sopra la `GameView` cinque menu sovrapposti (inizialmente nascosti o attivati in sequenza): `connectionMenu`, `charSelectionMenu`, `mapSelectionMenu`, `pauseMenu` e `disconnectMenu`, oltre all'etichetta diagnostica `fpsLabel`.

Il ciclo di vita dell'applicazione è scandito da un'istanza di `AnimationTimer` il cui metodo `handle(long now)` calcola il delta time reale $\Delta t_{\text{frame}} = \text{now} - \text{lastTime}$ e governa la macchina a stati principale:

1. **Controllo Emergenza Disconnessione (`handleControllerDisconnection`):** Ad ogni frame verifica tramite `inputManager.hasLostControllers(numPlayers)` che i pad richiesti siano ancora attivi. Se un pad si scollega durante la partita, attiva il flag `isDisconnected`, applica un filtro `GaussianBlur(25)` alla visuale di gioco, mostra il `disconnectMenu` rosso di emergenza e azzera gli accumulatori congelando istantaneamente il match fino alla riconnessione e conferma tramite tasto `X/A`.

2. **Stato Connessione Controller (`waitingForControllers`):** Attende che i giocatori premano un pulsante sui rispettivi gamepad per passare alla selezione dei lottatori.

3. **Stato Selezione Personaggi (`waitingForCharacterSelection`):** Permette ai due giocatori di muoversi sulla griglia `TilePane` con un tempo di debounce di 200 ms, cambiare la skin con i tasti `L1`/`R1`, confermare con `X` o annullare la scelta con `O`. Contemporaneamente, il metodo `updatePreviews()` istanzia due oggetti `Player` in modalità menu (`inMenuMode = true`) associati a due `PlayerRenderer` dedicati sul livello trasparente `previewLayer`, riproducendo in tempo reale l'animazione "MENU IDLE" con la skin selezionata.

4. **Stato Selezione Mappa (`waitingForMapSelection`):** Consente di scorrere le 10 arene in orizzontale e verticale, calcolando dinamicamente la posizione di scroll verticale `mapScrollPane.setVvalue(...)` per mantenere sempre centrata la miniatura selezionata e aggiornando lo sfondo reale della `GameView` in anteprima. Alla conferma (`confirmMapSelection()`), il controller estrae le due `CharacterFactory` scelte, ordina al Model di generare i lottatori (`model.spawnPlayers(...)`) iniettando anche gli indici di skin scelti (`skin1`, `skin2`), inizializza i renderer nella View e imposta il `groundLevelRatio` specifico della mappa.

5. **Stato di Gameplay e Combattimento (`handleGameplayState`):** Quando tutti i menu iniziali sono chiusi e il gioco non è in pausa, entra in funzione il **Game Loop con Accumulatore**. Il tempo trascorso Δt_{frame} viene sommato a `physicsAccumulator` e `renderAccumulator`:

   ```
   while (physicsAccumulator >= GameConfig.TIME_PER_TICK) {
       inputManager.update();
       model.update(inputManager);
       physicsAccumulator -= GameConfig.TIME_PER_TICK;
   }\
   ```

   Questo ciclo `while` garantisce che, se il computer subisce un rallentamento temporaneo, il motore isico esegua più passi discreti consecutivi di durata fissa `TIME_PER_TICK` per recuperare il tempo reale trascorso senza alterare la velocità percepita del gioco e senza causare tunnel geometrici nelle collisioni. Il rendering grafico viene invece eseguito solo quando `renderAccumulator` raggiunge la soglia targetFrameTime $= 10^9$/targetFps.

3.4.2 Gestione periferiche hardware (`InputManager`)

La classe `InputManager` è implementata secondo il pattern creazionale **Singleton** (`InputManager.getInstance()`) per garantire un unico punto di accesso centralizzato al sottosistema nativo `ControllerManager` della libreria *Jamepad* (SDL2) [3]. La classe mantiene gli indici fisici dei controller assegnati ai due giocatori (`p1Index` e `p2Index`, inizializzati a -1), gestisce l'accoppiamento automatico alla pressione dei tasti principali, applica una zona morta hardware del 15% (`Math.abs(stick) > 0.15`) sugli assi analogici per eliminare il fenomeno dello *stick drift* e fornisce metodi semantici ad alto livello per le azioni di combattimento:

- **Pugno standard (`isPunchButtonPressed`):** pressione del tasto `X` (Quadrato su layout PlayStation);

- **Gancio pesante (`isHookButtonPressed`):** combinazione del grilletto destro (rightTrigger > 0.5, tasto `R2`) + tasto `X`;

- **Mossa speciale Hadouken (`isHadoukenButtonPressed`):** combinazione del grilletto sinistro (leftTrigger > 0.5, tasto `L2`) + tasto `X`.

3.4.3 Gestione disaccoppiata del comparto audio (`SoundManager` e `SoundEvent`)

L'integrazione degli effetti sonori di combattimento (SFX) e delle colonne sonore di sottofondo (BGM) è stata progettata preservando la totale indipendenza del package `model` dal modulo multimediale `javafx.scene.media`. Il meccanismo si articola su tre componenti cooperanti:

1. **Dominio degli Eventi Sonori (`SoundEvent` e `soundQueue`):** All'interno di `pgsf.model` è definita l'enumerazione pura `SoundEvent` (`JUMP`, `PUNCH_SWING`, `HOOK_SWING`, `KICK_SWING`, `HADOUKEN`, `HIT`, `BLOCK`, `PARRY`, `KO`). La classe `GameModel` mantiene una coda FIFO `Queue<SoundEvent> soundQueue` (`ArrayDeque`). Quando la logica fisica valida l'esecuzione di un salto o di un attacco in `handleStandingActions()` e `handleCrouchingActions()`, oppure rileva un impatto, una parata parziale, un *Parry perfetto* o il *KO* in `handleCombat()`, `resolveMeleeBlockOrParry()`, `resolveProjectileHit()` e `checkGameOver()`, il modello non riproduce alcun file audio ma accoda l'evento semantico corrispondente (`soundQueue.add(...)`).

2. **Consumo Sincrono nel Game Loop (`GameController`):** All'interno del ciclo a passo fisso `handleGameplayState()`, immediatamente dopo l'aggiornamento fisico `model.update(inputManager)`, il controller preleva e svuota gli eventi generati nel tick corrente tramite il metodo `model.pollSoundEvents()`, delegandone la riproduzione effettiva a `SoundManager`:

   ```
   for (SoundEvent event : model.pollSoundEvents()) {
       SoundManager.getInstance().playSfx(event);
   }
   ```

3. **Sottosistema Multimediale con Caching (`SoundManager`):** Implementata secondo il pattern **Singleton** nel package `pgsf.utils`, la classe `SoundManager` adotta due strategie distinte in base alla natura del flusso audio:

   - *Effetti Sonori a Bassa Latenza (`AudioClip`):* I campioni brevi di combattimento e di interfaccia utente (`cursor`, `confirm`, `cancel`) vengono pre-caricati in RAM all'avvio dentro una `EnumMap<SoundEvent, AudioClip> sfxCache` e una `HashMap<String, AudioClip> uiSfxCache`. Ciò consente riproduzioni multiple sovrapposte con latenza nulla tramite `clip.play(sfxVolume)`.
   - *Musica di Sottofondo Persistente (`MediaPlayer`):* Le tracce musicali lunghe dei menu e delle arene vengono gestite in streaming tramite oggetti `MediaPlayer` configurati in ciclo continuo (`MediaPlayer.INDEFINITE`) e indicizzati in una mappa `Map<String, MediaPlayer> bgmCache`. Quando l'utente cambia schermata o mette in pausa l'incontro, la traccia precedente viene messa in stato `pause()` senza essere distrutta: in questo modo, tornando al menu o riprendendo la partita, la colonna sonora riparte esattamente dal secondo in cui era stata interrotta. Inoltre, i metodi `setBgmVolume()` e `setSfxVolume()` aggiornano in tempo reale il volume di tutte le istanze attive e in cache in risposta agli slider del menu impostazioni (`Settings`).

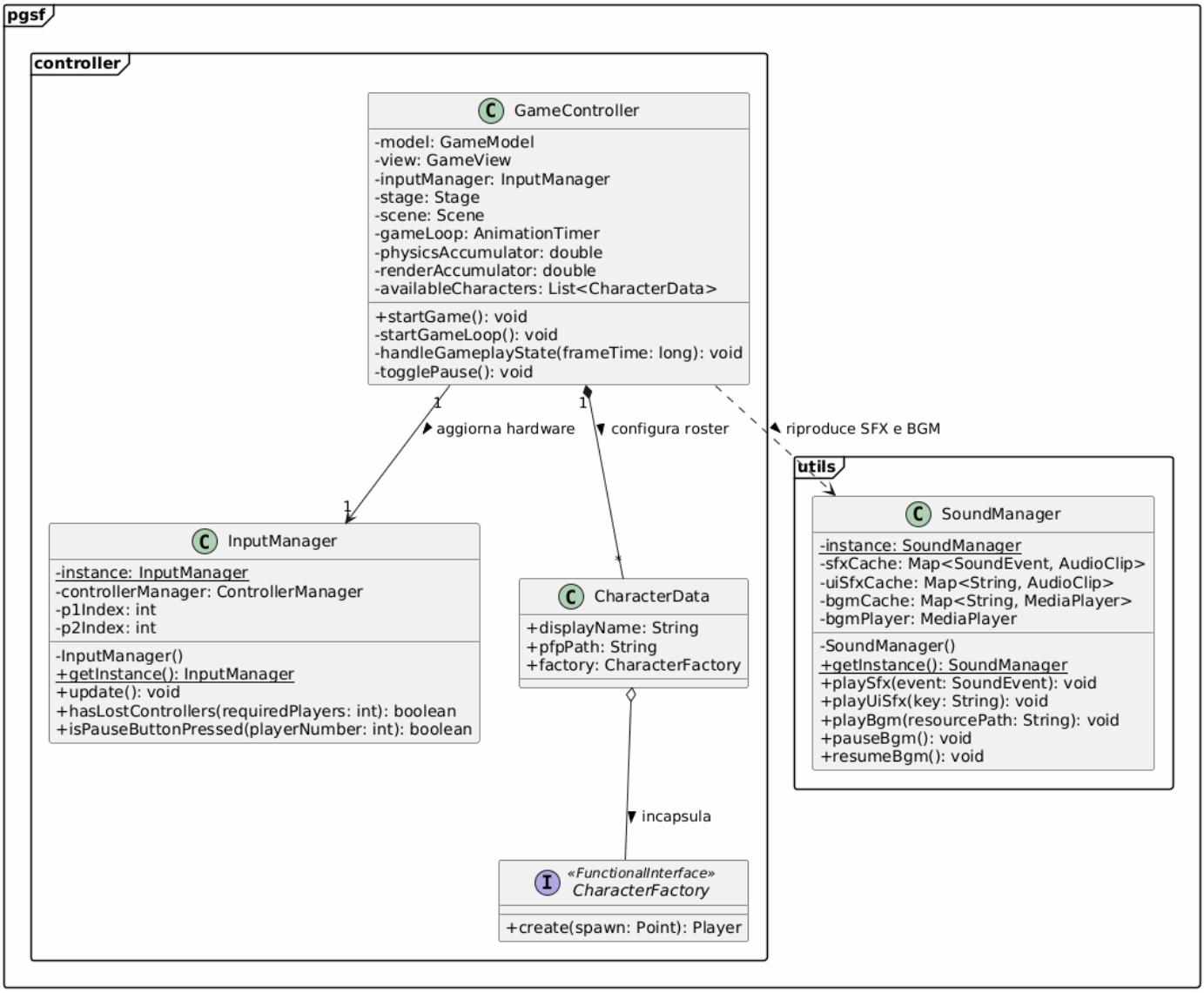


Figura 4: Dettaglio del diagramma delle classi per i package `pgsf.controller` e `pgsf.utils`.

3.5 Problemi riscontrati

Durante le fasi di sviluppo e integrazione dei moduli, sono emerse alcune problematiche architetturali e grafiche di rilievo, risolte mediante interventi mirati:

3.5.1 Avvio dell'applicazione tramite la classe `Main.java`

Uno dei problemi più significativi incontrati durante lo sviluppo ha riguardato l'avvio del programma. La classe `pgsf.Main`, che estende `javafx.application.Application` e ne implementa il metodo `start(Stage)`, era stata pensata come punto di ingresso dell'applicazione. Tuttavia, eseguendola direttamente, la Java Virtual Machine terminava subito con il seguente errore:

```
Error: JavaFX runtime components are missing, and are
required to run this application
```

Causa del problema. A partire da Java 11, JavaFX non è più incluso nel JDK, ma viene distribuito separatamente (progetto OpenJFX) sotto forma di *moduli* del Java Platform Module

System, come `javafx.graphics` e `javafx.controls`. Quando la classe indicata come principale estende `Application`, il launcher di Java esegue un controllo preliminare prima ancora di chiamare il metodo `main`: verifica che il modulo `javafx.graphics` sia presente nel *module path*. Se le librerie JavaFX si trovano invece nel normale *classpath*, come accade nella configurazione predefinita di molti IDE o quando le dipendenze vengono raccolte in un unico file JAR, il modulo non viene riconosciuto e l'avvio viene interrotto, anche se le classi necessarie sono effettivamente disponibili.

Soluzione adottata. Per aggirare il controllo abbiamo introdotto una classe `pgsf.Launcher`, che non estende `Application` e si limita a richiamare `Main.main(args)`. Poiché la classe principale non è più una sottoclasse di `Application`, il launcher di Java non effettua la verifica sui moduli; le classi di JavaFX vengono quindi caricate dal classpath come normali librerie e l'applicazione si avvia correttamente. Si tratta di una soluzione molto diffusa, ma non ufficialmente supportata: le versioni più recenti di JavaFX segnalano infatti questa configurazione con un avviso all'avvio (*Unsupported JavaFX configuration: classes were loaded from 'unnamed module'*), pur continuando a funzionare.

Soluzioni corrette. Il modo previsto per risolvere il problema è fornire esplicitamente i moduli JavaFX alla JVM. Le alternative principali sono:

- **Specificare il module path all'avvio**, indicando la cartella `lib` dell'SDK di JavaFX e i moduli da caricare:

  ```
  java --module-path /percorso/javafx-sdk/lib \
  --add-modules javafx.controls pgsf.Main
  ```

  Le stesse opzioni possono essere inserite negli argomenti della VM nella configurazione di esecuzione dell'IDE.

- **Rendere il progetto modulare**, aggiungendo un file `module-info.java` che dichiari la dipendenza dai moduli JavaFX (`requires javafx.controls;`) e renda accessibile il package principale a JavaFX (`opens pgsf to javafx.graphics;`).

- **Affidarsi a uno strumento di build**, come Maven o Gradle con i plugin ufficiali di OpenJFX (`javafx-maven-plugin` e `org.openjfx.javafxplugin`), che configurano automaticamente il module path.

- **Utilizzare un JDK che include già JavaFX**, come BellSoft Liberica JDK nella versione *Full* o Azul Zulu nella versione *FX*, nei quali i moduli sono presenti di default.

Per la distribuzione finale, inoltre, gli strumenti `jlink` e `jpackage` permettono di creare un'immagine di runtime personalizzata che contiene già i moduli JavaFX necessari, così da eliminare del tutto la dipendenza dalla configurazione della macchina dell'utente.

3.5.2 Rendering dello sprite sheet

L'implementazione del rendering di Ryu e Ken aveva due problemi principali: l'allineamento dei frame e l'efficienza del rendering di diverse immagini (una per frame). Tali criticità sono state risolte organizzando i frame in un'unica immagine (atlas) e associando un dizionario in cui ogni chiave univoca contiene le coordinate assolute nell'*atlas*, nonché l'altezza e la larghezza del singolo frame.

La classe `AnimData` fornisce i parametri logici dell'animazione (riga, colonna di partenza, numero totale di frame e velocità di transizione). Attraverso questi dati, il sistema individua

istantaneamente la chiave corretta senza calcoli geometrici complessi a runtime; un esempio di chiave è `"row00_col00_frame00"`.

Con la mappa pre-caricata in memoria (`HashMap<String, Rectangle2D>`) all'avvio dell'applicazione, il recupero del singolo frame avviene in tempo costante, azzerando i rallentamenti nel game loop.

3.5.3 Gestione del palette swap e iniezione delle skin

La personalizzazione cromatica dei lottatori ha richiesto di superare due criticità:

1. *Disallineamento tra selezione e spawn effettivo:* inizialmente, la classe `CharacterFactory` istanziava il personaggio con la palette predefinita, ignorando le scelte effettuate nel menu di selezione. Il problema è stato risolto estendendo il metodo `spawnPlayers(f1, f2, skin1, skin2)` all'interno del `GameModel`, consentendo al `GameController` di iniettare gli indici selezionati (`setSkinIndex`) prima della generazione dei renderer visivi.

2. *Invalidazione della cache delle texture nel renderer:* la classe `PlayerRenderer` utilizza una tabella hash (`frameCache`) per memorizzare i ritagli Nearest-Neighbor già calcolati. Durante la fase di anteprima nel menu, al cambio di skin tramite i dorsali `L1/R1`, il renderer rischiava di ripescare i fotogrammi già indicizzati della palette precedente. Si è quindi reso necessario forzare l'istanziazione di un nuovo `PlayerRenderer` dedicato ad ogni variazione di skin (`p1Changed || p2Changed`), garantendo il caricamento corretto del rispettivo file PNG con trasparenza cromatica (chromakey).

3.5.4 Disaccoppiamento della nascita dei proiettili (*Deferred Spawning*)

La generazione dell'*Hadouken* rischiava di produrre il proiettile all'istante di pressione del tasto anziché in corrispondenza del fotogramma in cui il lottatore estende le braccia. L'adozione della classe `RangedMove` dotata del flag `hasFired` ha consentito al loop di interrogare il metodo `createProjectile()` ad ogni tick e istanziare l'entità esattamente al completamento della finestra di *Startup*.

3.5.5 Sfarfallio e compenetrazione dei corpi nel pavimento

Durante la postura accovacciata (`crouch`), la riduzione dell'altezza della hurtbox provocava una sospensione aerea del modello visivo. La traslazione compensativa della coordinata Y ha consentito di mantenere i piedi dei combattenti ancorati al pavimento in qualsiasi transizione posturale.

4 Conclusioni e sviluppi futuri

L'adozione rigorosa del pattern **Model-View-Controller** e di interfacce di sola lettura (`IReadOnlyGameModel` e `IReadOnlyPlayer`) ha permesso di ottenere una base di codice modulare ed estendibile: l'integrazione di un nuovo lottatore o di una nuova arena richiede l'aggiunta di poche righe descrittive senza alterare il motore fisico o il sottosistema di rendering.

Tra i principali sviluppi futuri pianificati figurano:

- **Intelligenza Artificiale per Modalità Giocatore Singolo (CPU):** Implementazione di una macchina a stati decisionali (FSM) o di un albero di comportamento per consentire alla CPU di gestire reattivamente lo spacing, le parate su colpi in arrivo e le sequenze offensive di contrattacco;
- **Utilizzo della Barra del Potere Speciale (*Super Meter*):** Attivazione dinamica della barra energetica secondaria (già predisposta graficamente in `HUDView`) caricabile mediante parate riuscite o colpi a segno, utile per abilitare tecniche speciali avanzate;

5 Bibliografia

[1] Wikipedia. *Street Fighter (serie) – Wikipedia, l'enciclopedia libera*, 2026. [Online https://it.wikipedia.org/wiki/Street_Fighter_(serie)].

[2] OpenJFX Documentation. *JavaFX 17 / 21 Official API Documentation and Scene Graph Reference*, 2026. [Online https://openjfx.io/].

[3] Hartman, W. *Jamepad: A library for using gamepads in Java based on SDL2 GameController API*, GitHub Repository. [Online https://github.com/williamahartman/Jamepad].

[4] Nystrom, R. *Game Programming Patterns* (Capitolo *Game Loop – Fixed Time Step with Accumulator*). Genever Benning, 2014. [Online https://gameprogrammingpatterns.com/game-loop.html].

6 Appendice

In questa appendice vengono riportate le tabelle tecniche di riferimento relative alla mappatura dei comandi sui controller di gioco (`InputManager`), ai parametri delle arene (`MapData`) e alla configurazione dei fotogrammi di animazione (`AnimData`) dei lottatori.

6.1 Mappatura dei comandi controller (Gamepad)

La Tabella 1 riassume l'assegnazione dei comandi di gioco ed esplorazione dei menu implementata nella classe `InputManager` tramite le API SDL2 di Jamepad.

Input Hardware (PS / Xbox)	**Metodo `InputManager`**	**Azione nel Menu / In Partita**
Levetta Sinistra (Asse X/Y)	`getLeftStickX/Y()`	Navigazione griglie menu; Movimento orizzontale (`WALK`) e postura accovacciata (`CROUCH` per $Y > 0.5$).
Tasto Croce / A	`isJumpButtonPressed()`	Conferma selezione nei menu; Salto verticale (`JUMP`) con supporto *short hop*.
Tasto Cerchio / B	`isDefendButtonPressed()`	Annulla selezione personaggio; Parata in piedi (`BLOCK`), parata bassa (`CROUCH_BLOCK`) e *Parry*.
Tasto Quadrato / X	`isPunchButtonPressed()`	Pugno rapido (`PUNCH_RIGHT/LEFT`) o pugno basso (`PUNCH_CROUCH`).
Tasto Triangolo / Y	`isKickButtonPressed()`	Calcio (`KICK_RIGHT/LEFT`).
Grilletto R2/RT + Quadrato/X	`isHookButtonPressed()`	Gancio pesante (`HOOK_RIGHT/LEFT`).
Grilletto L2/LT + Quadrato/X	`isHadoukenButtonPressed()`	Attacco energetico a distanza (`HADOUKEN`).
Dorsali L1 / R1 (LB / RB)	`isL1/R1ButtonPressed()`	Scorrimento ciclico delle palette cromatiche (*skin*) nel menu personaggi.
Tasto Options / Start	`isPauseButtonPressed()`	Apertura e chiusura del menu di pausa in partita con filtro `GaussianBlur`.

Tabella 1: Mappatura completa degli input da controller gestita dalla classe `InputManager`.

6.2 Tabella dei frame data e statistiche dei lottatori

La Tabella 2 illustra la configurazione delle animazioni (`AnimData`) e dei parametri di combattimento definiti per i personaggi `Ryu` e `Ken`.

Stato animazione	**Riga atlas**	**Colonna iniziale**	**Frame totali**	**Durata frame (ms)**	**Loop**
`IDLE_RIGHT / LEFT`	0	0	4	100	Sì
`MENU_IDLE`	0	0	4	150	Sì
`WALK_RIGHT / LEFT`	0	1	5	75	Sì
`JUMP_RIGHT / LEFT`	0	2 / 3	7	80	No
`CROUCH_RIGHT / LEFT`	0	4	2	400	No
`BLOCK_RIGHT / LEFT`	0	5	1	50	No
`PUNCH_RIGHT` (Ryu / Ken)	1	0	3	40 / 80	No
`PUNCH_LEFT`	1	1	5	80	No
`HOOK_RIGHT / LEFT`	1	2 / 3	3 / 7	80	No
`KICK_LEFT / RIGHT` (Ryu)	2	0 / 1	3 / 5	40	No
`KICK_LEFT / RIGHT` (Ken)	2	0 / 1	3 / 5	80 / 75	No
`PUNCH_CROUCH`	3	0	3	40	No
`HADOUKEN` (Ryu / Ken)	5 / 4	0 / 5	5 / 4	80	No
`HURT_RIGHT / LEFT`	6	1	3	100	No
`HURT_CROUCH`	6	2	1	100	No

Tabella 2: Riepilogo dei parametri `AnimData` configurati nelle classi `Ryu` e `Ken`.